

ТОО “Колледж Хекслет”

КУРСОВАЯ РАБОТА

на тему: “Сайт по теме “Бортовой журнал обслуживания автомобиля””

по модулю: “Прикладное программирование”

Специальность: 06130100 Программное обеспечение (по видам)

Квалификация: 4S06130105 Техник информационных систем

Выполнил: студент 2 курса
12 ТИС/24 группы
Гальберг Михиал Евгеньевич

Допущен(-а) к защите “__” _____ 20__ г.

Преподаватель: _____ Турман Н.Р.

(подпись)

Защитил(-а) с оценкой:

(буква) (циф.экв.)

(балл)

Алматы 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ: ПРОЕКТИРОВАНИЕ ВЕБ-СЕРВИСА “БОРТОВОЙ ЖУРНАЛ ОБСЛУЖИВАНИЯ АВТОМОБИЛЯ”	5
1.1 Анализ предметной области веб-сервиса для проверки регламента ТО	5
1.2 Проектирование пользовательского интерфейса бортового журнала	6
1.3 Выбор и обоснование технологий разработки клиентской части	8
2 ПРАКТИЧЕСКАЯ ЧАСТЬ: РАЗРАБОТКА ВЕБ-СЕРВИСА “БОРТОВОЙ ЖУРНАЛ ОБСЛУЖИВАНИЯ АВТОМОБИЛЯ”	11
2.1 Реализация HTML-разметки веб-сервиса	11
2.2 Реализация CSS-стилей и адаптивного дизайна	15
2.3 Реализация JavaScript-функционала проверки регламентов по пробегу	19
ЗАКЛЮЧЕНИЕ	31
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33
ПРИЛОЖЕНИЕ	34

ВВЕДЕНИЕ

В условиях современного социально-экономического развития наблюдается устойчивый рост количества личных автотранспортных средств, что обусловлено повышением уровня урбанизации, доступности автомобилей и потребности населения в мобильности. Автомобиль в XXI веке выступает не только средством передвижения, но и важным элементом повседневной жизни, обеспечивающим независимость, экономию времени и комфорт. Однако увеличение автопарка неизбежно сопровождается ростом ответственности владельцев за техническое состояние своих транспортных средств.

Своевременное техническое обслуживание (ТО) автомобиля является ключевым фактором обеспечения безопасности дорожного движения и надежности эксплуатации транспортного средства. Регулярное проведение регламентных работ позволяет значительно продлить срок службы узлов и агрегатов, снизить вероятность возникновения аварийных ситуаций, а также минимизировать затраты на капитальный ремонт. Производители автомобилей, такие как Toyota и Hyundai, разрабатывают детализированные регламенты технического обслуживания, включающие интервалы замены расходных материалов и диагностики систем [10][11].

Несмотря на наличие подобных регламентов, на практике многие автовладельцы сталкиваются с проблемами их соблюдения. Традиционный формат ведения учета технического обслуживания в виде бумажных сервисных книжек постепенно утрачивает актуальность. Бумажные носители подвержены физическому износу, легко теряются, а поиск необходимой информации в них требует значительных временных затрат. Кроме того, ручной способ учета не обеспечивает автоматического напоминания о предстоящих работах, что увеличивает риск пропуска важных процедур обслуживания.

Исторически учет технического состояния автомобилей осуществлялся вручную: владельцы самостоятельно фиксировали данные о пробеге, рассчитывали интервалы замены моторного масла, фильтров и других расходных материалов, ориентируясь на показания одометра и рекомендации из печатных руководств. В ряде случаев информация о проведенных работах фиксировалась сотрудниками станций технического обслуживания в виде отметок или наклеек. Однако такой подход характеризуется низкой точностью, отсутствием систематизации и высокой зависимостью от человеческого фактора.

С развитием информационных технологий и глобальной цифровизации общества появилась возможность автоматизации подобных процессов. Современные веб-технологии позволяют создавать удобные цифровые сервисы, доступные пользователю в любой момент времени через браузер. В основе таких решений лежат стандартизированные технологии веб-разработки: язык разметки HTML, обеспечивающий структурирование контента [3], каскадные таблицы стилей CSS для визуального оформления интерфейса [2], а также язык программирования JavaScript, реализующий динамическую логику и обработку пользовательских действий [1]. Дополнительно взаимодействие элементов страницы и программной логики осуществляется через объектную модель документа (DOM), предоставляемую браузером [4].

На сегодняшний день рынок программного обеспечения предлагает различные решения для учета технического обслуживания автомобилей, включая мобильные приложения и веб-платформы. Однако большинство существующих систем характеризуются избыточной функциональностью, сложными интерфейсами, необходимостью регистрации и передачи персональных данных пользователя. Кроме того, многие сервисы ориентированы на коммерческую модель распространения, что ограничивает их доступность.

В то же время наблюдается выраженный спрос на легковесные и простые в использовании инструменты, не требующие установки дополнительного программного обеспечения. Особенно актуальными являются веб-сервисы, функционирующие непосредственно в браузере, поскольку они обеспечивают кроссплатформенность,

доступность и простоту использования. Разработка подобного решения позволяет устранить основные недостатки существующих систем и предоставить пользователю инструмент для быстрого получения информации о регламенте технического обслуживания.

Таким образом, объектом исследования в данной работе выступает процесс информатизации учета

технического обслуживания автотранспортных средств.

Предметом исследования является проектирование и разработка клиентской части веб-сервиса “Бортовой журнал обслуживания автомобиля”.

Целью работы является создание функционального, удобного и эргономичного веб-приложения, позволяющего пользователю на основе минимального набора входных данных (марка, модель и пробег автомобиля) автоматически получать актуальный перечень регламентных работ.

Для достижения поставленной цели были сформулированы следующие задачи:

- провести анализ технической документации производителей автомобилей и систематизировать данные о регламентах ТО;
- разработать алгоритмы обработки данных и фильтрации информации в зависимости от пробега транспортного средства;
- спроектировать пользовательский интерфейс с учетом современных требований UX/UI-дизайна;
- реализовать веб-приложение с использованием современных технологий фронтенд-разработки;
- обеспечить развертывание проекта на платформе GitHub Pages для свободного доступа пользователей [13].

В процессе выполнения работы использовались официальные технические документы автопроизводителей, а также специализированные ресурсы по веб-разработке, включая документацию MDN Web Docs. Разработка клиентской части велась с применением современных инструментов программирования, таких как Visual Studio Code [14], система контроля версий Git [8], а также платформа GitHub для хранения и публикации проекта [9].

Методологическую основу исследования составили методы системного анализа, классификации и синтеза информации, а также методы программного моделирования и прототипирования пользовательских интерфейсов, применяемые в современной практике фронтенд-разработки [5].

Дополнительно в процессе разработки использовались расширения среды Visual Studio Code, направленные на повышение продуктивности и качества кода. Применение инструментов для автодополнения HTML и CSS [18], локализации интерфейса среды разработки [19], а также визуализации цветовых значений [20] позволило оптимизировать процесс разработки и снизить вероятность ошибок при реализации пользовательского интерфейса.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ: ПРОЕКТИРОВАНИЕ ВЕБ-СЕРВИСА “БОРТОВОЙ ЖУРНАЛ ОБСЛУЖИВАНИЯ АВТОМОБИЛЯ”

1.1 Анализ предметной области веб-сервиса для проверки регламента ТО

Разработка любого программного продукта, в том числе веб-приложения, начинается с комплексного анализа предметной области, в рамках которого определяется круг решаемых задач, выявляются потребности пользователей и формируются функциональные требования к системе. В контексте создания веб-сервиса “Бортовой журнал обслуживания автомобиля” ключевой целью является разработка доступного, интуитивно понятного и функционального инструмента, предназначенного для автоматизированного контроля регламентного технического обслуживания транспортных средств.

Для достижения указанной цели в рамках анализа предметной области были сформулированы основные направления разработки. К ним относятся: проектирование эргономичного пользовательского интерфейса, реализация клиентской логики обработки и фильтрации данных в зависимости от текущего пробега автомобиля, а также обеспечение кроссплатформенной доступности сервиса без необходимости установки дополнительного программного обеспечения. Последний аспект особенно важен, поскольку современные веб-технологии позволяют реализовать подобный функционал непосредственно в браузере пользователя с использованием стандартов HTML [3], CSS [2] и JavaScript [1].

Особое внимание при анализе предметной области было уделено исследованию целевой аудитории. Основными пользователями разрабатываемого веб-сервиса выступают частные автовладельцы, начинающие водители и автомобильные энтузиасты, которые заинтересованы в самостоятельном контроле технического состояния своих транспортных средств. Данная категория пользователей регулярно сталкивается с необходимостью отслеживания сроков замены моторного масла, фильтров, тормозных колодок, свечей зажигания и других расходных элементов, регламентируемых производителями автомобилей [10][11].

Анализ пользовательского поведения показал, что основной проблемой является сложность поиска и интерпретации информации, содержащейся в объемных руководствах по эксплуатации. Кроме того, отсутствие централизованной системы учета увеличивает вероятность пропуска критически важных регламентных работ, что может привести к ускоренному износу деталей и повышенным затратам на ремонт. Следовательно, существует объективная потребность в разработке цифрового решения, способного предоставлять пользователю точные и структурированные рекомендации на основе минимального набора входных данных, таких как марка автомобиля и текущий пробег.

Для более глубокого понимания текущего состояния рынка был проведен анализ существующих программных решений. На сегодняшний день доступны различные платформы автомобильной тематики, включая тематические форумы, социальные сети и специализированные мобильные приложения. Однако большинство из них характеризуются рядом недостатков: перегруженность интерфейса, наличие рекламных интеграций, обязательная регистрация и необходимость передачи персональных данных. Кроме того, мобильные приложения требуют установки на устройство, занимают память и зачастую ограничивают функциональность бесплатной версии.

В отличие от существующих решений, разрабатываемый веб-сервис ориентирован на минимализм, быстроедействие и доступность. Он функционирует непосредственно в браузере пользователя, что исключает необходимость загрузки и установки дополнительного программного обеспечения. Использование стандартных веб-интерфейсов и взаимодействие с объектной моделью документа (DOM) [4] позволяет реализовать динамическое обновление данных и мгновенную реакцию системы на действия пользователя. —

На основании проведенного анализа были сформулированы ключевые функциональные требования к системе. Во-первых, веб-сервис должен обеспечивать возможность добавления одного или нескольких транспортных средств в виртуальное пространство (“Мой гараж”) с указанием базовых характеристик. Во-вторых, система должна автоматически анализировать введенные данные и выполнять фильтрацию базы регламентных работ, отображая только актуальные процедуры, соответствующие текущему пробегу автомобиля. В-третьих, интерфейс приложения должен быть адаптивным и корректно отображаться на

устройствах с различными характеристиками экранов – от настольных компьютеров до мобильных телефонов.

Дополнительно важным требованием является высокая интерактивность интерфейса. Это предполагает наличие динамических форм ввода, элементов управления и механизмов обратной связи, обеспечивающих удобство взаимодействия пользователя с системой. Реализация данных возможностей достигается за счет использования современных средств клиентской разработки, включая обработку событий и изменение структуры страницы через JavaScript [1].

Таким образом, проведенный анализ предметной области позволил определить основные направления разработки веб-сервиса, выявить потребности целевой аудитории и сформировать четкий набор функциональных требований. Полученные результаты являются фундаментом для последующего этапа проектирования пользовательского интерфейса и реализации программной логики системы.

1.2 Проектирование пользовательского интерфейса бортового журнала

Проектирование пользовательского интерфейса (User Interface, UI) и пользовательского опыта (User Experience, UX) является одним из ключевых этапов жизненного цикла разработки программного обеспечения. Именно на данном этапе формируется способ взаимодействия пользователя с системой, определяются визуальные и функциональные характеристики интерфейса, а также закладываются принципы удобства и эффективности работы с приложением. В рамках разработки веб-сервиса “Бортовой журнал обслуживания автомобиля” интерфейс выступает связующим звеном между сложной логикой обработки данных и конечным пользователем – автовладельцем.

Основной задачей проектирования являлось создание интуитивно понятной и минималистичной визуальной среды, способной снизить когнитивную нагрузку на пользователя и обеспечить быстрый доступ к необходимой информации. Для достижения этой цели процесс проектирования был структурирован на несколько последовательных этапов: разработка архитектуры интерфейса, создание прототипов (wireframes), проектирование навигационной системы, а также выбор цветовой палитры и типографики с учетом принципов визуальной иерархии и доступности.

Разработка структуры веб-приложения основывалась на принципах минимализма и функциональной целесообразности. Центральным элементом интерфейса стал модуль “Мой гараж”, представляющий собой основное рабочее пространство пользователя. Данный модуль обеспечивает управление данными о транспортных средствах и отображение информации о регламентном техническом обслуживании. Архитектура интерфейса была разделена на несколько логических зон: верхнюю информационную панель, основную рабочую область и зону взаимодействия с элементами управления.

В рабочей области реализован функционал отображения списка автомобилей с возможностью добавления новых записей. Также предусмотрен индикатор общего количества транспортных средств, позволяющий пользователю быстро оценить объем хранимых данных. Кнопка добавления нового автомобиля выступает в роли основного элемента управления и обеспечивает переход к форме ввода данных. Подобная

структурная организация интерфейса позволяет пользователю сосредоточиться на ключевых действиях без отвлечения на второстепенные элементы.

Организация контента внутри интерфейса реализована с учетом принципов наглядности и логической последовательности. Информация о регламентных работах представлена в виде структурированных списков, автоматически обновляемых в зависимости от введенных пользователем данных. Каждая карточка автомобиля содержит основные параметры: марку, модель, текущий пробег и статус обслуживания. Такой подход позволяет избежать необходимости перехода между страницами и обеспечивает сохранение контекста взаимодействия в рамках одного экрана.

На этапе прототипирования были разработаны детализированные макеты интерфейса (wireframes), позволяющие визуализировать структуру и поведение элементов до начала программной реализации. Для создания прототипов использовался специализированный инструмент Figma, который предоставляет широкие возможности для проектирования интерфейсов, работы с векторной графикой и создания переиспользуемых компонентов [7]. Применение данного инструмента позволило заранее определить расположение элементов управления, отступы, размеры и взаимосвязи компонентов интерфейса.

В процессе проектирования были созданы высокодетализированные макеты (High-Fidelity Wireframes), максимально приближенные к итоговому внешнему виду приложения. Дополнительно были разработаны уникальные графические элементы, включая иконки и логотип, что позволило сформировать визуальную идентичность продукта и повысить его узнаваемость.

Ключевым требованием к интерфейсу являлась адаптивность, обеспечивающая корректное отображение веб-сервиса на различных устройствах. Проектирование велось по принципу “mobile-first”, при котором первоначальное внимание уделяется мобильным устройствам, а затем интерфейс масштабируется под более крупные экраны. Это обусловлено тем, что пользователи чаще всего обращаются к подобным сервисам с мобильных устройств, находясь непосредственно в автомобиле или на станции технического обслуживания.

Все интерактивные элементы, включая кнопки, формы ввода и выпадающие списки, были спроектированы с учетом особенностей сенсорного управления. Размеры элементов и расстояния между ними подобраны таким образом, чтобы минимизировать вероятность ошибочных нажатий. При отображении на устройствах с большим экраном интерфейс адаптируется за счет перераспределения пространства, сохраняя при этом логическую структуру и удобство использования.

Особое внимание было уделено проектированию навигационной системы. Веб-сервис реализует плоскую архитектуру навигации, при которой пользователь достигает необходимого результата за минимальное количество действий. Отсутствие сложных иерархий и вложенных меню снижает порог входа и делает систему доступной для пользователей с различным уровнем цифровой грамотности. Пользовательский сценарий взаимодействия выстроен линейно: добавление автомобиля – ввод данных – получение рекомендаций по обслуживанию.

Дополнительно в интерфейсе реализованы элементы микро-взаимодействий, обеспечивающие визуальную обратную связь при взаимодействии с элементами управления. Это включает анимации кнопок, изменения состояния элементов при наведении и другие визуальные эффекты, повышающие понятность интерфейса и улучшающие пользовательский опыт.

Завершающим этапом проектирования стало формирование визуального стиля интерфейса, включающего выбор цветовой палитры и типографики. Для подбора цветовой схемы использовался специализированный инструмент Realtime Colors, позволяющий анализировать сочетание цветов и их контрастность [6]. Это обеспечило соответствие интерфейса современным требованиям доступности и улучшило читаемость информации.

В качестве основного шрифта был выбран Source Serif 4 – гарнитура, оптимизированная для цифровых интерфейсов. Использование шрифта с засечками обусловлено необходимостью повышения читаемости длинных текстовых блоков, таких как списки регламентных работ и технические описания. Наличие засечек способствует визуальному объединению символов в строке, что снижает зрительную нагрузку на пользователя. Кроме того, выбранная типографика придает интерфейсу более строгий и профессиональный вид, формируя доверие к предоставляемой информации.

Таким образом, комплексный подход к проектированию пользовательского интерфейса, включающий структурирование интерфейса, прототипирование в среде Figma [7], разработку адаптивной архитектуры и обоснованный выбор визуальных решений, позволил создать удобную и эффективную среду взаимодействия пользователя с веб-сервисом. Полученные результаты служат основой для дальнейшей реализации функциональной части приложения и обеспечивают высокий уровень удобства использования системы.

1.3 Выбор и обоснование технологий разработки клиентской части

Выбор технологического стека является одним из ключевых этапов разработки программного обеспечения, поскольку именно от него напрямую зависят такие характеристики системы, как производительность, масштабируемость, кроссбраузерная совместимость и удобство сопровождения исходного кода. В рамках проектирования клиентской части веб-сервиса “Бортовой журнал обслуживания автомобиля” приоритет был отдан созданию легковесного, быстрого и автономного решения, не зависящего от сторонних серверных компонентов и сложных программных инфраструктур.

На основании анализа требований к функциональности и производительности был выбран стек базовых веб-технологий, включающий HTML5, CSS3 и нативный JavaScript. Данный выбор обусловлен тем, что указанные технологии являются фундаментом современной веб-разработки и обеспечивают максимальную совместимость с различными браузерами и устройствами [5].

В качестве основного инструмента для формирования структуры веб-приложения был использован язык гипертекстовой разметки HTML5, предназначенный для логического и семантического описания содержимого веб-страницы [3]. В процессе разработки был применен подход семантической верстки, предполагающий использование специализированных тегов (например, header, nav, main, section, article и footer), позволяющих четко структурировать контент. Такой подход способствует улучшению доступности веб-приложения, облегчает его индексацию поисковыми системами и повышает читаемость исходного кода.

Дополнительным преимуществом HTML5 является наличие встроенных механизмов валидации пользовательских данных. Это позволяет реализовать первичную проверку корректности ввода (например, числовых значений пробега автомобиля) без необходимости написания дополнительного программного кода на стороне клиента.

Для реализации визуального оформления интерфейса были использованы каскадные таблицы стилей CSS3, обеспечивающие гибкую настройку внешнего вида элементов страницы [2]. В процессе разработки применялись современные модули CSS, такие как Flexbox и Grid Layout, позволяющие эффективно управлять расположением элементов в одномерном и двумерном пространстве соответственно. Flexbox использовался для выравнивания элементов навигации и управления, тогда как Grid Layout применялся для построения сеточной структуры карточек автомобилей и списков регламентных работ.

Адаптивность интерфейса достигалась за счет использования медиа-запросов CSS, позволяющих изменять параметры отображения в зависимости от характеристик

устройства пользователя. Это обеспечивает корректную работу веб-приложения как на настольных компьютерах, так и на мобильных устройствах.

С целью повышения модульности и удобства поддержки кода в проекте была использована методология БЭМ (блок–элемент–модификатор). Данный подход позволяет структурировать стили, разделяя интерфейс на независимые компоненты, что значительно упрощает масштабирование проекта и предотвращает конфликты между стилями.

Ключевую роль в реализации интерактивности и бизнес-логики веб-сервиса играет язык программирования JavaScript, поддерживающий мультипарадигменный подход к разработке [1]. В рамках проекта было принято решение использовать нативный JavaScript без привлечения сторонних библиотек и фреймворков. Это позволило минимизировать объем загружаемых ресурсов и повысить скорость работы приложения.

С помощью JavaScript реализованы основные функциональные возможности системы, включая обработку пользовательского ввода, динамическое обновление интерфейса и фильтрацию данных в зависимости от пробега автомобиля. Взаимодействие с элементами страницы осуществляется через объектную модель документа (DOM), которая предоставляет программный интерфейс для доступа и изменения структуры веб-документа [4].

Одним из важных архитектурных решений стало использование Local Storage API – встроенного механизма браузера для хранения данных на стороне клиента. Данный инструмент позволяет сохранять информацию о добавленных автомобилях (марка, модель, пробег и другие параметры) непосредственно в памяти браузера без

необходимости передачи данных на сервер. Это обеспечивает высокую скорость работы приложения, так как все операции выполняются локально, а также гарантирует конфиденциальность пользовательских данных.

Принцип работы Local Storage основан на хранении данных в формате “ключ–значение”, где значения сериализуются в строковый формат (например, JSON). При загрузке страницы данные могут быть извлечены, десериализованы и использованы для восстановления состояния интерфейса.

Важной частью процесса разработки является использование специализированных инструментов, повышающих эффективность работы программиста. В качестве основной среды разработки была выбрана Visual Studio Code – современный кроссплатформенный редактор кода, обладающий широкими возможностями расширения функционала за счет подключаемых модулей [14]. Данный инструмент обеспечивает подсветку синтаксиса, автодополнение кода, интеграцию с системой контроля версий и удобные средства навигации по проекту.

Для управления версиями исходного кода применялась распределенная система контроля версий Git, позволяющая отслеживать изменения, сохранять историю разработки и обеспечивать возможность отката к предыдущим версиям проекта [8][12]. В качестве удаленного репозитория использовалась платформа GitHub, предоставляющая удобные инструменты для совместной разработки и хранения проектов [9].

Дополнительно для публикации веб-приложения в сети Интернет использовался сервис GitHub Pages, позволяющий бесплатно размещать статические веб-сайты и обеспечивать их доступность для пользователей [13]. Это решение полностью соответствует концепции проекта как легковесного веб-сервиса, не требующего серверной части.

Для тестирования и отладки веб-приложения применялись встроенные инструменты разработчика браузера (DevTools), позволяющие анализировать структуру DOM, отслеживать выполнение скриптов, выявлять ошибки и оптимизировать производительность [17]. Использование данных инструментов значительно ускоряет процесс разработки и повышает качество итогового продукта.

Таким образом, выбранный технологический стек, основанный на использовании HTML [3], CSS [2] и JavaScript [1], а также современных инструментов разработки, таких

как Visual Studio Code [14], Git [8] и GitHub [9], полностью соответствует поставленным задачам проекта. Он обеспечивает высокую производительность, удобство использования, масштабируемость и простоту сопровождения веб-сервиса “Бортовой журнал обслуживания автомобиля”.

Дополнительно для повышения эффективности разработки и улучшения качества исходного кода в проекте использовались специализированные расширения среды разработки Visual Studio Code. В частности, расширение HTML CSS Support обеспечивает интеллектуальное автодополнение классов и стилей, что ускоряет процесс верстки и снижает вероятность синтаксических ошибок [18]. Использование русского языкового пакета интерфейса среды разработки способствует повышению удобства работы и снижению когнитивной нагрузки при взаимодействии с инструментами редактора [19]. Кроме того, расширение Color Highlight позволяет визуализировать цветовые значения непосредственно в коде, что значительно упрощает процесс работы с цветовой палитрой и повышает точность соответствия дизайну [20].

Для ускорения процесса разработки и оперативной проверки результатов верстки в проекте использовалось расширение Live Server для Visual Studio Code, позволяющее запускать локальный веб-сервер с функцией автоматической перезагрузки страницы при внесении изменений в код [15]. Данный инструмент значительно повышает эффективность разработки, так как устраняет необходимость ручного обновления страницы в браузере и обеспечивает мгновенную визуализацию изменений интерфейса. Это особенно важно при работе с HTML-разметкой и CSS-стилями, где требуется постоянный контроль за отображением элементов.

Для обеспечения единообразия и читаемости исходного кода в процессе разработки применялся автоматический форматировщик Prettier, интегрированный в среду Visual Studio Code [16]. Данный инструмент выполняет автоматическое выравнивание структуры кода, расстановку отступов, перенос строк и приведение синтаксиса к единому стилю. Использование Prettier позволяет минимизировать количество стилистических ошибок, облегчает сопровождение проекта и делает код более понятным как для самого разработчика, так и для других участников разработки.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ: РАЗРАБОТКА ВЕБ-СЕРВИСА “БОРТОВОЙ ЖУРНАЛ ОБСЛУЖИВАНИЯ АВТОМОБИЛЯ”

2.1 Реализация HTML-разметки веб-сервиса

Для обеспечения масштабируемости и удобства поддержки веб-ресурса была спроектирована логическая файловая структура. Четкое распределение ресурсов по тематическим каталогам позволяет оптимизировать процесс разработки и упростить навигацию по компонентам приложения. На рисунке 1 представлена визуальная иерархия каталогов и файлов в среде разработки.

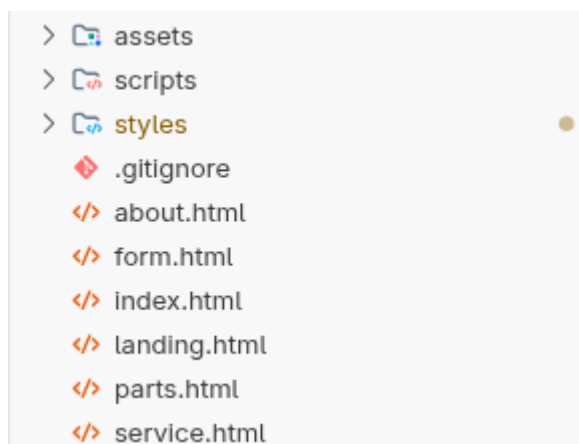


Рисунок 1 – Файловая структура проекта

Примечание – составлено автором на основании проведенного исследования

Организация проекта предполагает разделение статических ресурсов, скриптов и стилевого оформления. Папка assets предназначена для хранения мультимедийных файлов и шрифтов, scripts содержит исполняемые файлы JavaScript, а в каталоге styles группируются таблицы стилей. В корневом разделе размещен файл .gitignore для настройки системы контроля версий, а также набор HTML-документов, формирующих каркас страниц веб-приложения. Схематичное представление данной иерархии с использованием специальных символов выглядит следующим образом:

```
root/
├── assets/
├── scripts/
├── styles/
├── .gitignore
├── about.html
├── form.html
├── index.html
├── landing.html
├── parts.html
└── service.html
```

В основе визуального представления главной страницы веб-приложения лежит HTML-разметка, определяющая ключевые элементы пользовательского интерфейса. Для настройки корректного отображения и подключения внешних зависимостей была сформирована заголовочная часть документа (рисунок 2). В данном фрагменте кода определяются метаданные страницы, настраивается область просмотра (viewport) для обеспечения адаптивности на мобильных устройствах, а также подключаются каскадные таблицы стилей (CSS) и внешние шрифтовые наборы.

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Car Service Tracker - Упрощаем уход за авто</title>
  <link rel="stylesheet" href="/styles/normalize.css">
  <link rel="stylesheet" href="/styles/style.css">
  <link rel="stylesheet" href="/styles/index.css">

  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
    href="https://fonts.googleapis.com/css2?
family=Source+Serif+4:ital,opsz,wght@0,8..60,200..900;1,8..60,200..900&display=swap"
    rel="stylesheet">
</head>

```

Рисунок 2 – Структура заголовочной части документа (тег head)

Примечание – составлено автором на основании проведенного исследования

Основным элементом взаимодействия пользователя с системой является интерфейс предварительной записи на техническое обслуживание. Разработанная форма включает сгруппированные элементы управления, которые для удобства рассмотрения и оформления в соответствии с требованиями разделены на несколько структурных блоков.

Первая часть формы (рисунок 3) предназначена для ввода персональных данных клиента, таких как ФИО и контактный номер телефона, а также выбора желаемой даты посещения сервиса с помощью встроенного календаря.

```

<form action="service.html" method="GET">
  <div class="form-group">
    <label for="fullName">ФИО</label>
    <input type="text" id="fullName" name="fullName" placeholder="Иванов Иван
Иванович" required>
  </div>

  <div class="form-row">
    <div class="form-group">
      <label for="phone">Номер телефона</label>
      <input type="tel" id="phone" name="phone" placeholder="+7 (999)
000-00-00" required>
    </div>
    <div class="form-group">
      <label for="date">Дата записи</label>
      <input type="date" id="date" name="date" required>
    </div>
  </div>

```

Рисунок 3 – Фрагмент разметки формы записи: ввод персональных данных

Примечание – составлено автором на основании проведенного исследования

Вторая часть (рисунок 4) отвечает за сбор информации о транспортном средстве. Пользователь указывает марку, модель, текущий пробег в числовом формате и выбирает тип необходимого обслуживания (плановое ТО, ремонт или диагностика) из выпадающего списка.

```

<div class="form-row">
  <div class="form-group">
    <label for="brand">Марка авто</label>
    <input type="text" id="brand" name="brand" placeholder="например, Toyota"
required>
  </div>
  <div class="form-group">
    <label for="model">Модель</label>
    <input type="text" id="model" name="model" placeholder="например, Camry"
required>
  </div>
</div>

<div class="form-row">
  <div class="form-group">
    <label for="mileage">Текущий пробег (км)</label>
    <input type="number" id="mileage" name="mileage" placeholder="например,
55000" required>
  </div>
  <div class="form-group">
    <label for="serviceType">Тип обслуживания</label>
    <select id="serviceType" name="serviceType" required>
      <option value="">Выберите тип...</option>
      <option value="routine">Плановое ТО</option>
      <option value="repair">Ремонт</option>
      <option value="diagnostics">Диагностика</option>
    </select>
  </div>
</div>

```

7

Рисунок 4 – Фрагмент разметки формы записи: данные об автомобиле и типе услуг
Примечание – составлено автором на основании проведенного исследования

Заключительный блок формы (рисунок 5) содержит многострочное текстовое поле для дополнительных замечек или пожеланий клиента, а также кнопку отправки заполненных данных (submit) на обработку.

```

<div class="form-group mb-25">
  <label for="notes">Заметки</label>
  <textarea id="notes" name="notes" rows="3"
placeholder="Дополнительные детали о проблемах или пожелания..."></
textarea>
</div>

<button type="submit" class="btn btn-primary w-100">Записаться</button>
</form>

```

7

Рисунок 5 – Фрагмент разметки формы записи: дополнительные заметки и отправка данных
Примечание – составлено автором на основании проведенного исследования

Для отображения ключевой информации о состоянии виртуального гаража пользователя предусмотрен специальный информационный блок (рисунок 6). Он содержит заголовки страницы с кратким описанием раздела и элемент статистики со счетчиком “Всего машин”, значение которого предназначено для динамического обновления.

```

<div class="dashboard-header">
  <div>
    <h1>Мой гараж</h1>
    <p class="lead mb-0">Управление расписанием ТО и запчастями.</p>
  </div>

  <div class="flex-center-card">
    <div class="icon-box-primary">
      </div>
    <div>
      <div class="text-xs-light-upper">
        Всего машин</div>
      <div class="text-xl-bold" id="total-cars">0</div>
    </div>
  </div>
</div>

```

Рисунок 6 – Разметка информационного блока панели управления
Примечание – составлено автором на основании проведенного исследования

С целью обеспечения удобной работы и быстрого доступа к нужным записям в интерфейс интегрирована панель фильтрации (рисунок 7). Данный блок включает в себя текстовое поле для сквозного поиска автомобилей по марке, а также элементы управления для вызова функционала фильтров и быстрого перехода к форме добавления нового транспортного средства.

```

<div class="filters-bar mb-30">
  <input type="text" id="search-input" class="search-input"
    placeholder="Фильтр по марке (например, BMW, Toyota)">
  <button class="btn btn-outline flex-center-gap-8">
    Фильтры
  </button>
  <a href="index.html#register" class="btn btn-primary flex-center-gap-8">
    Добавить
  </a>
</div>

```

Рисунок 7 – Разметка панели поиска и фильтрации
Примечание – составлено автором на основании проведенного исследования

Вывод информации о зарегистрированных автомобилях организован в виде адаптивной сеточной структуры (рисунок 8). Изначально в контейнере размещена стационарная карточка-кнопка, инициирующая процесс добавления нового транспорта. В этот же контейнер посредством JavaScript в дальнейшем будут выводиться карточки добавленных автомобилей.

```

<div class="grid" id="garage-grid">
  <a href="index.html#register" class="card add-card">
    <div class="add-icon">
      </div>
    <h3 class="text-lg-bold-mb-10">
      Добавить новый транспорт</h3>
    <p class="text-sm-light">Отслеживайте обслуживание для еще одного
      автомобиля в гараже.</p>
  </a>
</div>

```

Рисунок 8 – Контейнер сеточной структуры для вывода карточек автомобилей
Примечание – составлено автором на основании проведенного исследования

Для обеспечения интерактивности страницы в нижней части документа располагается блок подключения клиентских сценариев (рисунок 9). Данные файлы содержат модульную логику веб-приложения: управление визуальными темами, взаимодействие с локальным хранилищем данных браузера, вспомогательные функции и основной исполняемый код страницы.

```
<script src="./scripts/theme.js"></script>
<script src="./scripts/storage.js"></script>
<script src="./scripts/utils.js"></script>
<script src="./scripts/main.js"></script>
```

Рисунок 9 – Подключение файлов клиентских сценариев (JavaScript)

Примечание – составлено автором на основании проведенного исследования

2.2 Реализация CSS-стилей и адаптивного дизайна

Визуальное оформление веб-приложения задается с помощью каскадных таблиц стилей (CSS). Для обеспечения гибкости интерфейса и удобного управления цветовой палитрой в корневом селекторе определены пользовательские CSS-переменные для светлой темы (рисунок 10). Данный подход позволяет централизованно задавать цвета текста, фона, теней и акцентных элементов.

```
@import url('https://fonts.googleapis.com/css2?
family=Source+Serif+4:ital,opsz,wght@0,8..60,200..900;1,8..60,200..900&display=swap')
;

/* Светлая тема (по умолчанию и через атрибут) */
:root,
:root[data-theme="light"] {
  --text: #241200;
  --text-light: #522900;
  --text-dark: #000000;

  --background: #e6e6e6;
  --background-light: #ffffff;
  --background-dark: #d9d9d9;

  --primary: #ff5757;
  --primary-light: #ff8585;
  --primary-dark: #d94a4a;

  --secondary: #d5a47b;
  --secondary-light: #e4c1a4;
  --secondary-dark: #b58b68;

  --accent: #719d3f;
  --accent-light: #8dbd4f;
  --accent-dark: #5a7d32;

  --card-shadow: rgba(0, 0, 0, 0.08);
}
```

Рисунок 10 – Определение CSS-переменных для светлой темы оформления

Примечание – составлено автором на основании проведенного исследования

Для реализации темной темы используются два механизма (рисунок 11): медиа-запрос `@media (prefers-color-scheme: dark)`, который автоматически применяет стили в зависимости от системных настроек пользователя, и селектор атрибута

[data-theme="dark"] для принудительного переключения темы через JavaScript. В обоих случаях значения базовых переменных переопределяются на более темные оттенки.

```
/* Темная тема (через медиа-запрос) */
@media (prefers-color-scheme: dark) {
  :root {
    --text: #ffeddb;
    --text-light: #ffffff;
    --text-dark: #d9caba;

    --background: #0f0f0f;
    --background-light: #262626;
    --background-dark: #000000;

    --primary: #a80000;
    --primary-light: #d40000;
    --primary-dark: #7d0000;

    --secondary: #84522a;
    --secondary-light: #a66735;
    --secondary-dark: #633e20;

    --accent: #94c062;
    --accent-light: #abd37f;
    --accent-dark: #789c4f;

    --card-shadow: rgba(0, 0, 0, 0.5);
  }
}

:root[data-theme="dark"] {
  /* Переопределение переменных аналогично медиа-запросу */
  --text: #ffeddb;
  --text-light: #ffffff;
  --text-dark: #d9caba;
  --background: #0f0f0f;
  --background-light: #262626;
  --background-dark: #000000;
  --primary: #a80000;
  --primary-light: #d40000;
  --primary-dark: #7d0000;
  --secondary: #84522a;
  --secondary-light: #a66735;
  --secondary-dark: #633e20;
  --accent: #94c062;
  --accent-light: #abd37f;
  --accent-dark: #789c4f;
  --card-shadow: rgba(0, 0, 0, 0.5);
}
```

Рисунок 11 – Переопределение переменных для темной темы оформления
Примечание – составлено автором на основании проведенного исследования

Глобальные настройки стилей, представленные на рисунке 12, направлены на создание стабильной основы для построения макета. Применение универсального селектора со свойством box-sizing: border-box позволяет избежать проблем с расчетом

фактической ширины элементов при добавлении внутренних отступов и границ. В блоке определены параметры для тела документа, включая выбор основной гарнитуры и настройку межстрочного интервала, что способствует улучшению читаемости контента. Описание класса контейнера ограничивает максимальную ширину контента и обеспечивает его автоматическое центрирование, создавая единообразные боковые поля на экранах различного разрешения.

```
* {  
    box-sizing: border-box;  
    margin: 0;  
    padding: 0;  
}  
  
body {  
    font-family: 'Source Serif 4', serif;  
    color: var(--text);  
    background-color: var(--background);  
    line-height: 1.6;  
    display: flex;  
    flex-direction: column;  
    min-height: 100vh;  
}  
  
.container {  
    max-width: 1200px;  
    margin: 0 auto;  
    padding: 0 20px;  
}
```

1

Рисунок 12 – Базовые стили сброса, типографики и основного контейнера
Примечание – составлено автором на основании проведенного исследования

Стилизация компонентов управления строится на принципах модульности и расширяемости, что наглядно демонстрирует рисунок 13. Базовый класс для кнопок содержит общие параметры геометрии, шрифта и курсора, а также определяет длительность и характер анимационных переходов. Отдельный класс для первичных кнопок задает акцентную заливку, выделяя основные целевые действия. Реализация псевдокласса для состояния наведения включает в себя не только изменение цвета фона, но и трансформацию элемента с добавлением мягкой тени, что создает эффект интерактивности и улучшает пользовательский опыт за счет визуального отклика на действия автора.

```

.btn {
  display: inline-block;
  padding: 10px 24px;
  border-radius: 8px;
  font-weight: 600;
  text-decoration: none;
  text-align: center;
  cursor: pointer;
  transition: all 0.3s ease;
  border: none;
}

.btn-primary {
  background-color: var(--primary);
  color: #fff;
}

.btn-primary:hover {
  background-color: var(--accent);
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(255, 87, 87, 0.3);
}

```

Рисунок 13 – Стилизация кнопок и эффектов наведения

Примечание – составлено автором на основании проведенного исследования

Как показано ниже на рисунке 14, первая часть кода содержит базовую Flexbox-структуру секции .hero и медиа-запросы для обеспечения адаптивности на мобильных устройствах.

```

.hero {
  display: flex;
  justify-content: space-between;
  gap: 40px;
  margin-bottom: 4rem;
}

@media (max-width: 768px) {
  .hero {
    flex-direction: column;
  }
}

```

Рисунок 14 – Стилизация основного контейнера секции Hero

Примечание – составлено автором на основании проведенного исследования

2.3 Реализация JavaScript-функционала проверки регламентов по пробегу

Для реализации функционала веб-приложения были разработаны вспомогательные скрипты. В частности, для форматирования текста и приведения его к нужному регистру используется специальная функция, представленная на рисунке 15.

```
function formatTitle(str) {  
    if (!str) return '';  
    return String(str).split(' ').map(word => Math.max(word.length) > 0 ?  
word[0].toUpperCase() + word.substr(1) : '').join(' ');  
}
```

7

Рисунок 15 – Вспомогательная функция formatTitle для форматирования регистра текста
Примечание – составлено автором на основании проведенного исследования

Данная функция принимает строковое значение, разбивает его на массив слов по пробелу, и для каждого элемента переводит первую букву в верхний регистр, оставляя остальные без изменений, после чего объединяет их обратно в единую строку. Это позволяет корректно отображать заголовки и названия в интерфейсе пользователя.

Для предотвращения мигания не стилизованного контента при загрузке страницы реализована немедленная инициализация темы оформления (рисунок 16).

```
(function() {  
    const savedTheme = localStorage.getItem('theme');  
    if (savedTheme) {  
        document.documentElement.setAttribute('data-theme', savedTheme);  
    } else {  
        const prefersDark = window.matchMedia && window.matchMedia('(prefers-color-  
scheme: dark)').matches;  
        document.documentElement.setAttribute('data-theme', prefersDark ? 'dark' :  
'light');  
    }  
})();
```

Рисунок 16 – Самовызывающаяся функция инициализации темы приложения
Примечание – составлено автором на основании проведенного исследования

Представленный скрипт проверяет наличие ранее сохраненной темы в локальном хранилище браузера. Если значение отсутствует, система автоматически определяет предпочтения пользователя на уровне операционной системы с помощью метода window.matchMedia и устанавливает соответствующий атрибут для корневого элемента документа.

Управление данными автомобилей пользователя осуществляется через функции взаимодействия с локальным хранилищем браузера, алгоритм которых продемонстрирован на рисунке 17.

```

function getData() {
    try {
        const raw = localStorage.getItem('savedCars');
        if (raw) return JSON.parse(raw);
    } catch (e) {
        console.error('Failed to parse cars from localStorage', e);
    }
    return [];
}

function saveData(data) {
    localStorage.setItem('savedCars', JSON.stringify(data));
}

```

Рисунок 17 – Функции getData и saveData для работы с локальным хранилищем

Примечание – составлено автором на основании проведенного исследования

Функция получения данных извлекает строку из хранилища и преобразует ее в объект с помощью встроенного метода. В случае возникновения ошибки при преобразовании, в консоль выводится соответствующее сообщение, а функция возвращает пустой массив, что предотвращает сбои в работе приложения. Функция сохранения выполняет обратную операцию, конвертируя переданные данные в формат текстовой строки для записи в память браузера.

Для реализации механизма “умного поиска”, устойчивого к опечаткам со стороны пользователя, в веб-приложении был применен алгоритм вычисления расстояния Левенштейна. На рисунке 18 представлена базовая конструкция функции для оценки сходства строк.

```

// Расстояние Левенштейна для умного поиска
function getLevenshteinDistance(a, b) {
    if (a.length === 0) return b.length;
    if (b.length === 0) return a.length;

    const matrix = [];
    for (let i = 0; i <= b.length; i++) {
        matrix[i] = [];
    }
    for (let j = 0; j <= a.length; j++) {
        matrix[0][j] = j;
    }

    for (let i = 1; i <= b.length; i++) {
        for (let j = 1; j <= a.length; j++) {
            if (b.charAt(i - 1) === a.charAt(j - 1)) {
                matrix[i][j] = matrix[i - 1][j - 1];
            } else {
                matrix[i][j] = Math.min(
                    matrix[i - 1][j - 1] + 1, // замена
                    Math.min(
                        matrix[i][j - 1] + 1, // вставка
                        matrix[i - 1][j] + 1 // удаление
                    )
                );
            }
        }
    }
    return matrix[b.length][a.length];
}

```

Рисунок 18 – Функция getLevenshteinDistance для вычисления расстояния между строками

Примечание – составлено автором на основании проведенного исследования

Данный программный модуль позволяет математически определить степень различия между двумя строками. Алгоритм вычисляет минимальное количество односимвольных операций (вставок, удалений или замен), необходимых для превращения одной последовательности в другую, что служит базой для обработки неточного текстового ввода.

Для применения описанного математического алгоритма на практике была реализована функция поиска наилучшего совпадения в имеющейся коллекции данных, структура которой продемонстрирована на рисунке 19.

```
// Поиск наиболее подходящего ключа (с допущением опечаток)
function findBestMatch(input, collection) {
    if (!input) return null;
    let bestKey = null;
    let minDistance = Infinity;

    const keys = Array.isArray(collection) ? collection :
    Object.keys(collection);

    for (let key of keys) {
        // Если точное или частичное совпадение
        if (key.includes(input) || input.includes(key)) {
            return key;
        }

        // Иначе считаем расстояние Левенштейна
        const distance = getLevenshteinDistance(input, key);

        // Допускаем опечатки (примерно 1-3 символа в
        // зависимости от длины)
        let threshold = Math.floor(Math.max(input.length,
        key.length) / 3);
        if (threshold < 1) threshold = 1;

        if (distance <= threshold && distance < minDistance) {
            minDistance = distance;
            bestKey = key;
        }
    }
    return bestKey;
}
```

Рисунок 19 – Функция findBestMatch для поиска наиболее подходящего ключа
Примечание – составлено автором на основании проведенного исследования

Представленная функция принимает введенный пользователем запрос и последовательно сравнивает его с доступными ключами в системе. В случае допущения пользователем неточностей при вводе, система оценивает вычисленную дистанцию между словами, обновляет значение минимальной дистанции и в итоге возвращает наиболее релевантный результат, что существенно повышает эргономичность пользовательского интерфейса.

В качестве локальной базы данных для хранения информации о регламентном техническом обслуживании автомобилей используется структурированный объект. Фрагмент инициализации константы продемонстрирован на рисунке 20.

```
document.addEventListener('DOMContentLoaded', () => {
  // --- 1. БАЗА ДАННЫХ ---
  const serviceData = {
    "mitsubishi": {
      "outlander": [
        { "mileage": 15000, "name": "TO 1 (15 000 км)",
      "parts": [ ... ] }
      ]
    }
  };
};
```

7

Рисунок 20 – Инициализация локальной базы данных serviceData

Примечание – составлено автором на основании проведенного исследования

Представленная структура включает в себя марку автомобиля, конкретную модель, а также массив регламентных работ с указанием требуемого пробега и перечня необходимых запасных частей. Для каждой детали предусмотрены минимальный и максимальный пороги стоимости, что позволяет системе в дальнейшем автоматически рассчитывать приблизительную смету на техническое обслуживание в зависимости от выбранного транспортного средства.

Дополнительно в рамках разработки был реализован функциональный модуль расчета ориентировочной стоимости технического обслуживания. Данный модуль выполняет роль встроенного калькулятора, позволяющего пользователю на основе выбранного автомобиля и текущего пробега получить приблизительную смету предстоящих работ.

Алгоритм расчета базируется на использовании локальной структуры данных serviceData, в которой каждая регламентная операция содержит диапазон стоимости (минимальное и максимальное значение). В процессе выполнения вычислений происходит фильтрация массива работ по условию соответствия текущему пробегу транспортного средства. Далее для каждой подходящей операции извлекаются значения стоимости и агрегируются в общий диапазон. Формально расчет может быть представлен следующей зависимостью:

- Общая минимальная стоимость: сумма всех минимальных значений операций;
- Общая максимальная стоимость: сумма всех максимальных значений операций.

Результат вычислений динамически выводится в интерфейс пользователя посредством изменения содержимого DOM-элементов. Обновление происходит в реальном времени при изменении входных параметров, что обеспечивает высокую интерактивность системы и удобство анализа данных.

Реализация данного функционала значительно повышает практическую ценность веб-сервиса, позволяя пользователю не только отслеживать регламентные работы, но и оценивать потенциальные финансовые затраты на обслуживание автомобиля.

Связывание программной логики с элементами пользовательского интерфейса осуществляется строго после полного построения объектной модели документа браузером. Инициализация ключевых элементов раздела “Мой гараж” продемонстрирована на рисунке 21.

```
document.addEventListener('DOMContentLoaded', () => {
  const grid = document.getElementById('garage-grid');
  const searchInput = document.getElementById('search-input');
  const totalCarsEl = document.getElementById('total-cars');
});
```

7

Рисунок 21 – Инициализация переменных для работы с DOM-элементами страницы

Примечание – составлено автором на основании проведенного исследования

В рамках данного скрипта посредством встроенного метода `getElementById` осуществляется поиск и сохранение в константы ссылок на сетку вывода карточек добавленных автомобилей, поле ввода поискового запроса и счетчик общего количества транспортных средств. Выполнение данного кода внутри обработчика события `DOMContentLoaded` гарантирует, что все элементы HTML-разметки уже загружены и доступны для дальнейших манипуляций.

Для обеспечения интерактивности пользовательского интерфейса, в частности, для возможности переключения визуальной темы оформления, реализован механизм поиска соответствующих элементов управления. Фрагмент кода, отвечающий за поиск кнопок смены темы, представлен на рисунке 22.

```
document.addEventListener('DOMContentLoaded', () => {
  const themeToggleBtns = document.querySelectorAll('.theme-toggle');
  if (!themeToggleBtns.length) return;

  themeToggleBtns.forEach(btn => {
    btn.removeAttribute('title');
    btn.addEventListener('click', () => {
      const currentTheme = document.documentElement.getAttribute('data-theme');
      const newTheme = currentTheme === 'dark' ? 'light' : 'dark';
      document.documentElement.classList.add('theme-transitioning');
      document.documentElement.setAttribute('data-theme', newTheme);
      localStorage.setItem('theme', newTheme);
      setTimeout(() => document.documentElement.classList.remove('theme-
transitioning'), 500);
    });
  });
});
```

Рисунок 22 – Инициализация элементов управления для переключения темы оформления
Примечание – составлено автором на основании проведенного исследования

Данный скрипт активируется только после полной загрузки объектной модели документа браузером. С помощью метода `querySelectorAll` в константу собираются все элементы, содержащие класс переключателя темы. Важным архитектурным решением является встроенная проверка на наличие данных элементов: если искомые кнопки отсутствуют в разметке текущей страницы, выполнение функции немедленно прерывается с помощью оператора `return`. Подобный подход позволяет избежать ошибок исполнения скрипта на тех страницах, где функционал смены темы не предусмотрен.

Обработка данных, вводимых пользователем на главной странице приложения, требует поэтапного программного взаимодействия с формой регистрации. Первым шагом является поиск необходимого элемента в структуре документа и назначение обработчика событий, что продемонстрировано на рисунке 23.

```
document.addEventListener('DOMContentLoaded', () => {
  const registerForm = document.querySelector('#register form');

  if (registerForm) {
    registerForm.addEventListener('submit', (e) => {
      e.preventDefault();
    });
  }
});
```

Рисунок 23 – Инициализация формы регистрации и перехват события отправки
Примечание – составлено автором на основании проведенного исследования

Данный участок кода выполняет целевой поиск элемента формы. Условие проверки наличия формы позволяет избежать ошибок на страницах, где она отсутствует. При

попытке отправки данных срабатывает метод предотвращения стандартного поведения браузера, что отменяет автоматическую перезагрузку страницы и позволяет обработать информацию асинхронно.

После перехвата события осуществляется сбор данных из соответствующих текстовых полей. Процесс получения значений и их базовая валидация представлены на рисунке 24.

```
const brandInput = document.getElementById('brand');
const modelInput = document.getElementById('model');
const mileageInput = document.getElementById('mileage');
const fullNameInput = document.getElementById('fullName');
const phoneInput = document.getElementById('phone');
const dateInput = document.getElementById('date');
const serviceTypeInput = document.getElementById('serviceType');
const notesInput = document.getElementById('notes');

if (!brandInput || !modelInput || !mileageInput) return;

const brand = brandInput.value.trim();
const model = modelInput.value.trim();
const mileage = mileageInput.value.trim();

if (!brand || !model || !mileage) return;
```

7

Рисунок 24 – Сбор данных из полей ввода и проверка на заполненность
Примечание – составлено автором на основании проведенного исследования

Скрипт осуществляет привязку переменных к элементам интерфейса по их уникальным идентификаторам. Далее реализована система защиты от сохранения пустых или некорректных данных: программа проверяет существование ключевых полей (марка, модель, пробег) и очищает введенные строки от лишних пробелов по краям. В случае отсутствия обязательной информации выполнение функции прерывается.

Прошедшие проверку данные структурируются в единый объект для удобства дальнейшей обработки. Структура создаваемого объекта транспортного средства показана на рисунке 9.

```
const newCar = {
  id: Date.now(),
  brand: brand,
  model: model,
  mileage: parseInt(mileage, 10),
  fullName: fullNameInput ? fullNameInput.value.trim() : '',
  phone: phoneInput ? phoneInput.value.trim() : '',
  date: dateInput ? dateInput.value.trim() : '',
  serviceType: serviceTypeInput ? serviceTypeInput.value : '',
  notes: notesInput ? notesInput.value.trim() : '',
  status: false
};
```

Рисунок 25 – Формирование объекта с данными транспортного средства
Примечание – составлено автором на основании проведенного исследования

Формируемый объект содержит исчерпывающую информацию о регистрируемом автомобиле и владельце. В качестве уникального идентификатора записи используется текущее системное время в миллисекундах. Строковое значение пробега конвертируется в целочисленный формат с помощью встроенной функции, а для необязательных полей применяются тернарные операторы, предотвращающие запись неопределенных значений.

Заключительным этапом алгоритма является сохранение сформированной записи и маршрутизация пользователя в соответствующий раздел приложения. Данный процесс приведен на рисунке 26.

```
// Get existing cars
const cars = window.getData ? window.getData() : [];

// Add new car
cars.push(newCar);

// Save to localStorage
if (window.saveData) {
    window.saveData(cars);
}

// Redirect to garage
window.location.href = 'index.html';
});
```

Рисунок 27 – Сохранение информации в локальное хранилище и перенаправление

Примечание – составлено автором на основании проведенного исследования

На данном этапе происходит обращение к глобальным вспомогательным функциям для получения массива уже существующих автомобилей. Новый объект добавляется в конец полученного массива, после чего обновленный список передается в функцию сохранения для записи в локальное хранилище браузера. Завершается выполнение скрипта автоматическим перенаправлением пользователя на главную страницу гаража.

Для обеспечения визуальной привлекательности раздела гаража и корректного отображения карточек транспортных средств, у которых отсутствует загруженная пользователем фотография, была разработана вспомогательная функция автоматического подбора изображений. Данный программный модуль представлен на рисунке 28.

```
function getCarImage(id) {
    const images = [
        'https://images.unsplash.com/photo-1555215695-3004980ad54e?
auto=format&fit=crop&q=80&w=800',
        'https://images.unsplash.com/photo-1621007947382-bb3c3994e3fd?
auto=format&fit=crop&q=80&w=800',
        'https://images.unsplash.com/photo-1606152421802-db97b9c7a11b?
auto=format&fit=crop&q=80&w=800',
        'https://images.unsplash.com/photo-1552519507-da3b142c6e3d?
auto=format&fit=crop&q=80&w=800'
    ];
    return images[id % images.length];
}
```

7

Рисунок 29 – Функция getCarImage для назначения изображений автомобилям

Примечание – составлено автором на основании проведенного исследования

Представленная функция содержит массив ссылок на заранее подготовленные тематические изображения высокого качества. Алгоритм принимает в качестве аргумента уникальный числовой идентификатор автомобиля и с помощью математической операции получения остатка от деления выбирает индекс из массива. Подобный подход гарантирует, что за конкретным транспортным средством всегда будет закреплена одна и та же картинка даже после перезагрузки страницы.

Для обеспечения удобной навигации по добавленным автомобилям была реализована функция их отрисовки со встроенным механизмом фильтрации. Начало алгоритма функции рендеринга и логика поиска продемонстрированы на рисунке 30.

```
function renderCars(filterText = '') {
  const lowerFilter = filterText.toLowerCase();

  let filteredCars = cars;
  if (lowerFilter) {
    filteredCars = cars.filter(car =>
      (car.brand && car.brand.toLowerCase().includes(lowerFilter)) ||
      (car.model && car.model.toLowerCase().includes(lowerFilter))
    );
  }
}
```

Рисунок 30 – Реализация алгоритма фильтрации массива автомобилей

Примечание – составлено автором на основании проведенного исследования

Алгоритм принимает строку поискового запроса и предварительно переводит ее в нижний регистр во избежание конфликтов чувствительности к регистру. Далее применяется встроенный метод фильтрации массива: система перебирает все сохраненные объекты автомобилей и проверяет наличие искомой подстроки в полях марки или модели. Результат записывается в новую локальную переменную, которая в дальнейшем используется для генерации разметки.

После формирования отфильтрованного списка транспортных средств осуществляется обновление статистических показателей интерфейса и подготовка базовых элементов управления. Этот процесс отражен на рисунке 31.

```
if (totalCarsEl) {
  totalCarsEl.textContent = filteredCars.length;
}

// Очищаем сетку, кроме add-card
const addCardHtml = `
  <a href="form.html" class="card add-card">
    <div class="add-icon">
      <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
linejoin="round"><path d="M12 5v14M5 12h14"/></svg>
    </div>
    <h3 class="text-lg-bold-mb-10">Добавить новый транспорт</h3>
    <p class="text-sm-light">Отслеживайте обслуживание для еще одного автомобиля
в гараже.</p>
  </a>
`;
```

Рисунок 31 – Обновление счетчика машин и формирование карточки добавления

Примечание – составлено автором на основании проведенного исследования

На данном этапе система проверяет наличие элемента счетчика в объектной модели документа и динамически обновляет его текстовое содержимое, подставляя актуальную

длину отфильтрованного массива. Сразу после этого в локальную константу записывается строковый шаблон HTML-кода для статической карточки добавления нового автомобиля. Эта карточка выступает в роли ссылки на форму регистрации и будет всегда отображаться в интерфейсе гаража первой, независимо от результатов поисковой выдачи.

Для визуального отображения списка автомобилей в интерфейсе раздела “Мой гараж” используется динамическая генерация HTML-разметки на основе полученных данных. Процесс формирования структуры карточки начинается с подготовки заголовка и вывода изображения, что продемонстрировано на рисунке 32.

```
filteredCars.forEach(car => {
  const title = window.formatTitle ? (window.formatTitle(car.brand) + ' ' +
    window.formatTitle(car.model)) : (car.brand + ' ' + car.model);

  html += `
    <div class="card" data-id="${car.id}">
      <div class="card-img">
        

```

Рисунок 32 – Начальный этап динамической генерации карточки автомобиля
Примечание – составлено автором на основании проведенного исследования

Алгоритм перебирает массив отфильтрованных транспортных средств в цикле. Для каждого элемента предварительно формируется заголовок с использованием функции приведения слов к верхнему регистру. Далее инициализируется добавление кода в строковую переменную разметки: создается контейнер карточки с уникальным идентификатором, внутри которого размещается автоматически подобранное изображение и специальный бейдж, отражающий текущий статус обслуживания.

Основной информационный блок содержит сведения о пробеге и детальные данные о владельце. Программная реализация заполнения данного раздела показана на рисунке 33.

```
<h3 class="card-title">${title}</h3>
<div class="card-meta" style="display: flex; justify-content: space-between; align-
items: center;">
  <span>${car.mileage.toLocaleString('ru-RU')} км</span>
  <button class="btn-edit-mileage" style="font-size: 0.8rem; padding: 4px 8px;
border: 1px solid var(--border); border-radius: 4px; background: transparent; cursor:
pointer;">Редактировать пробег</button>
</div>

<div class="card-details" style="margin-top: 15px; font-size: 0.9rem; color: var(--
text-light); line-height: 1.4;">
  ${car.fullName ? `<p style="margin: 4px 0;"><strong>Владелец:</strong>
${car.fullName}</p>` : ''}
  ${car.phone ? `<p style="margin: 4px 0;"><strong>Телефон:</strong> ${car.phone}</
p>` : ''}
  ${car.date ? `<p style="margin: 4px 0;"><strong>Дата записи:</strong>
${car.date.split('-').reverse().join('.')}</p>` : ''}
  ${car.serviceType ? `<p style="margin: 4px 0;"><strong>Тип:</strong>
${(routine:'Плановое ТО', repair:'Ремонт', diagnostics:'Диагностика')}
[car.serviceType] || car.serviceType}</p>` : ''}
  ${car.notes ? `<p style="margin: 4px 0;"><strong>Заметки:</strong> ${car.notes}</
p>` : ''}
</div>
```

Рисунок 33 – Формирование информационного блока с деталями обслуживания
Примечание – составлено автором на основании проведенного исследования

В данном фрагменте осуществляется форматирование числового значения пробега в соответствии с региональными стандартами с помощью встроенного метода

локализации, что повышает удобство восприятия информации пользователем. Дополнительная информация, такая как контактный телефон клиента, дата записи и тип требуемых работ, интегрируется в разметку с использованием тернарных операторов. Это обеспечивает условный вывод: элементы создаются только в том случае, если соответствующие поля были заполнены при регистрации транспортного средства. При этом дата записи программно преобразуется из стандартного системного формата в традиционный вид с точками.

Завершающим этапом генерации карточки является формирование элементов управления, логика которых приведена на рисунке 34.

```
<div class="txt-xs-light-upper-mb-10" style="margin-top: 15px;">Статус:</div>
<div class="tags">
  <span class="tag" style="background: ${car.status ? 'var(--accent)' : 'var(--background-dark)'}; color: ${car.status ? '#000' : 'var(--text-light)'}">
    ${car.status ? 'В порядке' : 'Требуется внимания'}
  </span>
</div>
<div class="card-footer" style="margin-top: 15px;">
  <label class="checkbox-label ${car.status ? 'txt-accent' : ''}">
    <input type="checkbox" class="status-checkbox" ${car.status ? 'checked' : ''}
  > Выполнено
  </label>
  <button class="icon-btn btn-delete" title="Удалить">
    <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
    stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
    linejoin="round"><polyline points="3 6 5 6 21 6"></polyline><path d="M19 6l-2 14H7L5
    6"></path><path d="M10 11v6"></path><path d="M14 11v6"></path><path d="M9 6V4a2 2 0 0
    1 2-2h2a2 2 0 0 1 2 2v2"></path></svg>
  </button>
</div> </div> </div>`;}
```

Рисунок 34 – Генерация тегов статуса и кнопок управления карточкой

Примечание – составлено автором на основании проведенного исследования

Нижняя часть карточки содержит визуальный тег, цветовое оформление и текст которого динамически меняются в зависимости от логического статуса объекта в базе данных. Интерактивная функциональность обеспечивается наличием флажка-переключателя для отметки о завершении сервисных работ и специальной кнопки с векторной иконкой (SVG) для полного удаления записи. Сгенерированная таким образом полная строка разметки добавляется к общей переменной для последующей интеграции в объектную модель документа.

После успешной генерации HTML-разметки и вывода карточек автомобилей на экран, системе необходимо обеспечить их интерактивность. Для этого в конце функции рендеринга вызывается специальный метод назначения обработчиков событий, первый блок которого отвечает за удаление транспортного средства (рисунок 35).

```
function attachEvents() {
  // Удаление
  document.querySelectorAll('.btn-delete').forEach(btn => {
    btn.addEventListener('click', (e) => {
      const card = e.target.closest('.card');
      const id = parseInt(card.dataset.id, 10);
      if (confirm('Вы уверены, что хотите удалить этот автомобиль?')) {
        cars = cars.filter(c => c.id !== id);
        window.saveData(cars);
        renderCars(searchInput ? searchInput.value : '');
      }
    });
  });
}
```

Рисунок 35 – Программная реализация логики удаления карточки автомобиля

Примечание – составлено автором на основании проведенного исследования

Данный участок кода находит все кнопки удаления на странице и привязывает к ним функцию отслеживания клика. При нажатии алгоритм определяет, к какой именно карточке относится кнопка, извлекая ее уникальный идентификатор. В целях предотвращения случайной потери информации система вызывает встроенное диалоговое окно подтверждения. Если пользователь подтверждает действие, массив данных фильтруется, удаленный объект исключается из списка, после чего обновленная информация сохраняется в локальное хранилище и интерфейс перерисовывается.

Для поддержания актуальности данных о техническом состоянии предусмотрена функция быстрого редактирования текущего пробега транспортного средства. Программный код данного механизма продемонстрирован на рисунке 36.

```
// Редактирование пробега
document.querySelectorAll('.btn-edit-mileage').forEach(btn => {
  btn.addEventListener('click', (e) => {
    const card = e.target.closest('.card');
    const id = parseInt(card.dataset.id, 10);
    const idx = cars.findIndex(c => c.id === id);
    if (idx !== -1) {
      const newMileageStr = prompt(`Введите новый пробег для
${window.formatTitle(cars[idx].brand)} ${window.formatTitle(cars[idx].model)}:`,
cars[idx].mileage);
      if (newMileageStr !== null) {
        const newMileage = parseInt(newMileageStr.trim(), 10);
        if (!isNaN(newMileage) && newMileage >= 0) {
          cars[idx].mileage = newMileage;
          window.saveData(cars);
          renderCars(searchInput ? searchInput.value : '');
        } else {
          alert('Некорректный пробег!');
        }
      }
    }
  });
});
```

Рисунок 36 – Функция обновления показаний пробега с валидацией ввода

Примечание – составлено автором на основании проведенного исследования

При активации функции редактирования система осуществляет поиск индекса нужного автомобиля в локальном массиве. Затем на экран выводится диалоговое окно ввода, в котором уже предзаполнено текущее значение пробега, а в заголовке окна динамически подставляются отформатированные марка и модель машины. Введенное пользователем значение проходит строгую валидацию: программа удаляет лишние пробелы, преобразует строку в число и проверяет, чтобы оно не было отрицательным или пустым. В случае ошибки ввода пользователю демонстрируется предупреждение, а при успешной проверке новые данные записываются в память.

Заключительная часть скрипта отвечает за изменение статуса обслуживания, а также за инициализацию поиска и первоначальную загрузку данных при открытии страницы. Реализация этих процессов представлена на рисунке 37.

```

// Чекбокс statyca
document.querySelectorAll('.status-checkbox').forEach(checkbox => {
  checkbox.addEventListener('change', (e) => {
    const card = e.target.closest('.card');
    const id = parseInt(card.dataset.id, 10);
    const idx = cars.findIndex(c => c.id === id);
    if (idx !== -1) {
      cars[idx].status = e.target.checked;
      window.saveData(cars);
      renderCars(searchInput ? searchInput.value : '');
    });});
// Слушатель на фильтр
if (searchInput) {
  searchInput.addEventListener('input', (e) => {
    renderCars(e.target.value);
  });}
// Инициализация
renderCars();});

```

Рисунок 37 – Обработка статуса технического обслуживания и активация поиска
Примечание – составлено автором на основании проведенного исследования

Обработчик изменения состояния чекбокса реагирует на клик пользователя, мгновенно обновляя логический статус автомобиля в базе данных. Вынесенный за пределы функции назначения событий слушатель поля ввода активирует поиск: при вводе каждого нового символа он автоматически вызывает функцию перерисовки интерфейса, передавая в нее введенный текст, что обеспечивает мгновенную фильтрацию списка без необходимости перезагрузки страницы. Скрипт завершается первичным вызовом функции рендеринга, которая отображает сохраненные данные сразу после полной загрузки документа.

В процессе практической реализации веб-приложения важную роль сыграли инструменты автоматизации разработки. Использование расширения Live Server обеспечило мгновенную проверку изменений в структуре и внешнем виде страниц без необходимости ручного обновления браузера [15]. Одновременно применение форматировщика Prettier позволило поддерживать единый стиль оформления кода, что повысило его читаемость и упростило дальнейшую доработку и сопровождение проекта [16].

ЗАКЛЮЧЕНИЕ

В рамках выполнения данной курсовой работы была рассмотрена актуальная задача разработки веб-сервиса, предназначенного для автоматизации учета технического обслуживания автотранспортных средств. Актуальность выбранной темы обусловлена ростом количества личных автомобилей и необходимостью повышения уровня ответственности автовладельцев за техническое состояние своих транспортных средств. В условиях цифровизации общества переход от устаревших бумажных методов учета к современным веб-решениям является логичным и востребованным направлением развития.

В ходе выполнения работы были последовательно решены все поставленные задачи. На первом этапе был проведен детальный анализ предметной области, в результате которого выявлены основные проблемы пользователей, связанные с отслеживанием регламентов технического обслуживания. Установлено, что существующие решения либо перегружены функционалом, либо требуют регистрации и передачи персональных данных, что снижает их удобство и доступность. Это позволило обосновать необходимость создания легковесного веб-сервиса, ориентированного на простоту и скорость использования.

На этапе проектирования пользовательского интерфейса были применены современные подходы UX/UI-дизайна, направленные на минимизацию когнитивной нагрузки пользователя и повышение удобства взаимодействия с системой. Разработана логическая структура веб-приложения, включающая модуль “Мой гараж” как центральный элемент управления данными. Были созданы прототипы интерфейса, продумана система навигации и подобрана цветовая схема, обеспечивающая хорошую читаемость и визуальную иерархию элементов. Особое внимание уделено адаптивности интерфейса, что позволяет использовать сервис на различных устройствах, включая мобильные телефоны.

Выбор технологического стека был осуществлен с учетом требований к производительности, простоте реализации и независимости от сторонних сервисов. В качестве базовых технологий использованы HTML, CSS и JavaScript, что позволило создать кроссплатформенное веб-приложение, работающее непосредственно в браузере. Применение нативного JavaScript без использования сторонних библиотек обеспечило высокую скорость работы и минимальный объем загружаемых ресурсов. Для хранения данных был использован Local Storage, что позволило реализовать автономную работу сервиса без необходимости подключения к серверу и обеспечило конфиденциальность пользовательской информации.

Практическая часть работы включала разработку полноценного веб-приложения. Была реализована логическая структура проекта с разделением на модули, отвечающие за разметку, стилизацию и функциональность. Создана HTML-разметка страниц, обеспечивающая корректную структуру интерфейса и удобство взаимодействия пользователя с системой. Реализованы CSS-стили, включая поддержку светлой и темной темы оформления, а также адаптивный дизайн, корректно отображающийся на различных устройствах.

Особое внимание было уделено разработке клиентской логики на языке JavaScript. Реализованы функции работы с локальным хранилищем данных, обеспечивающие сохранение и загрузку информации о транспортных средствах. Разработан механизм добавления, удаления и редактирования данных автомобилей, а также реализована система динамической отрисовки интерфейса. Для повышения удобства поиска применен алгоритм Левенштейна, позволяющий обрабатывать пользовательский ввод с учетом возможных ошибок и опечаток. Это значительно улучшает пользовательский опыт и делает систему более устойчивой к некорректному вводу данных.

Дополнительно в процессе разработки использовались современные инструменты

программирования, включая Visual Studio Code и его расширения, систему контроля версий Git и платформу GitHub. Это позволило организовать эффективный процесс разработки, обеспечить контроль изменений и подготовить проект к публикации. Развертывание веб-приложения на платформе GitHub Pages обеспечило его доступность в сети Интернет без необходимости использования серверной инфраструктуры.

В результате выполнения курсовой работы был разработан функциональный веб-сервис “Бортовой журнал обслуживания автомобиля”, который полностью соответствует поставленной цели. Созданное приложение позволяет пользователю быстро и удобно получать информацию о необходимых регламентных работах на основе введенных данных, что способствует повышению эффективности контроля технического состояния автомобиля.

Практическая значимость работы заключается в возможности реального применения разработанного веб-сервиса в повседневной жизни автовладельцев. Приложение может быть использовано как основа для дальнейшего развития и масштабирования, включая добавление серверной части, интеграцию с базами данных автопроизводителей, а также реализацию системы уведомлений и расширенного функционала учета.

Таким образом, можно сделать вывод, что поставленная цель работы достигнута, все задачи успешно выполнены, а полученные результаты подтверждают целесообразность использования современных веб-технологий для решения задач автоматизации учета технического обслуживания автомобилей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Mozilla. (2024). JavaScript. MDN WebDocs.
<https://developer.mozilla.org/ru/docs/Web/JavaScript>
2. Mozilla. (2024). CSS: Cascading Style Sheets. MDN WebDocs.
<https://developer.mozilla.org/en-US/docs/Web/CSS>
3. Mozilla. (2024). HTML: HyperText Markup Language. MDN WebDocs.
<https://developer.mozilla.org/ru/docs/Web/HTML>
4. Mozilla. (2024). HTML DOM API. MDN WebDocs.
https://developer.mozilla.org/en-US/docs/Web/API/HTML_DOM_API
5. Hexlet. (2024). Курс “Фронтенд-разработчик”. “Хекслет”.
<https://ru.hexlet.io/programs/frontend>
6. Realtime Colors. (2024). Visualize your color palettes. Realtime Colors.
<https://www.realtimecolors.com/>
7. Figma. (2024). Figma Design Documentation. Figma.
<https://help.figma.com/>
8. Git. (2026). Git. Git-SCM.
<https://git-scm.com/>
9. GitHub. (2024). GitHub Documentation. GitHub.
<https://docs.github.com/>
10. Toyota. (2023). Регламент технического обслуживания автомобилей. Toyota.
<https://www.toyota.ru/>
11. Hyundai. (2023). Руководство по эксплуатации и регламент ТО. Hyundai.
<https://www.hyundai.ru/>
12. Chacon, S., & Straub, B. (2024). Pro Git. Git-SCM.
<https://git-scm.com/book/en/v2>
13. GitHub. (2024). GitHub Pages Documentation. GitHub.
<https://docs.github.com/en/pages>
14. Microsoft. (2024). Visual Studio Code Docs. Visual Studio Code.
<https://code.visualstudio.com/docs>
15. Ritwick, D. (2024). Live Server Extension. Visual Studio Marketplace.
<https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer>
16. Prettier. (2024). Prettier - Code formatter. Visual Studio Marketplace.
<https://marketplace.visualstudio.com/items?itemName=esbenp.prettier-vscode>
17. Google. (2024). Chrome DevTools Documentation. Chrome Developers.
<https://developer.chrome.com/docs/devtools/>
18. ecmel. (2026). HTML CSS Support. Open VSX Registry.
<https://open-vsx.org/extension/ecmel/vscode-html-css>
19. Microsoft. (2026). Russian Language Pack for Visual Studio Code. Open VSX Registry.
<https://open-vsx.org/extension/MS-CEINTL/vscode-language-pack-ru>
20. naumovs. (2026). Color Highlight. Open VSX Registry.
<https://open-vsx.org/extension/naumovs/color-highlight>

